

NeuroDiscover (Quorum)

Pipeline & Infrastructure Guide

Part I: Plain-language overview (no coding background required)

Part II: Technical summary for developers

Generated from project docs: docs/workflow/, agent-io.md, design.md

Part 1

Plain-Language Overview

For readers with no coding experience. Uses analogies instead of implementation detail.

What NeuroDiscover Does

NeuroDiscover automates commercial drug-discovery research. Instead of experts spending months manually reading papers and debating opportunities, six specialized AI assistants (agents) run in sequence. Each agent reads what previous agents wrote in a shared database (the blackboard) and adds its own findings.

The goal: find hidden patient subgroups inside a disease, connect them to biological mechanisms and treatments, score scientific and commercial strength, and output ranked recommendations (Prioritize / Monitor / Reject).

The Problem It Solves

- Subgroup inside a heterogeneous disease - portfolio tools often aggregate at indication level.
- Cross-condition mechanism links - therapeutic areas are reviewed in silos.
- Emerging evidence after the last review cycle - manual synthesis does not scale.

The Blackboard (Shared Database)

Agents do not call each other directly. They read and write SQL tables:

Table	Plain meaning
evidence	One card per paper, trial, or grant
subgroups	Official patient segment definitions
treatment_connections	Subgroup -> mechanism -> treatment links
connection_evidence	Which sources support each link
recommendations	Final ranked advice
agent_outputs	Audit log per pipeline run

The Six Agents

Agent 1 - Literature Synthesis (The Research Librarian)

Searches PubMed, bioRxiv (optional), ClinicalTrials.gov, and NIH grants. Extracts structured fields: subgroup, mechanism, treatment, key_result. Writes evidence rows and links them to seed subgroups when names match exactly.

- Input: external scientific APIs (BioMCP, RePORTER)
- Output: evidence, subgroup_evidence, scan_state, agent_outputs

Agent 2 - Patient Subgroup (The Patient Classifier)

Groups evidence by subgroup label and writes canonical definitions to subgroups. Example demo names:

GBA-mutation PD, LRRK2 PD, Alpha-synuclein-high PD.

- Input: evidence rows with subgroup populated
- Output: subgroups, agent_outputs

Agent 3 - Treatment Connection (The Matchmaker)

Builds triples: patient subgroup + biological mechanism + treatment. Counts supporting sources and records which papers/trials back each link.

- Input: subgroups + evidence with mechanism and treatment
- Output: treatment_connections, connection_evidence, agent_outputs

Agent 4 - Evidence Scoring (The Skeptic)

Scores how convincing each connection is (0-100). Considers evidence count and source types: literature, trials (stronger), grants. Scores update when new papers land.

- Input: connections from Agent 3 + linked evidence
- Output: treatment_connections.evidence_strength, agent_outputs

Agent 5 - Commercial Discovery (The Business Analyst)

Estimates commercial potential (0-100) using heuristics: evidence volume, subgroup keywords (GBA, LRRK2), mechanism class, treatment presence.

- Input: scored connections from Agent 4
- Output: treatment_connections.commercial_potential, agent_outputs

Agent 6 - Conclusion Update (The Decision Maker)

Combines scores into confidence and tiers. Formula: $\text{confidence} = \text{evidence_strength} \times 0.55 + \text{commercial_potential} \times 0.45$.

- Prioritize: confidence ≥ 80
- Monitor: $65 \leq \text{confidence} < 80$
- Reject: confidence < 65
- Output: recommendations, agent_outputs

Optional CUA package (cua/): separate, heavier Agent 6 that writes an NIH grant proposal to local files. Read-only on DB; not part of dashboard pipeline.

Infrastructure (Four Layers)

- Database - SQLite locally or Supabase Postgres (team shared, ~307 evidence rows live).
- Orchestrator - Runs agents 1-6 in order; shared run_id; commits after each agent.
- API server - FastAPI reads tables; serves dashboard JSON.
- Dashboard - Single-page UI at <http://127.0.0.1:8080>; polls agent_outputs for live progress.

Run Modes

Mode	Behavior	When
demo	Offline/sample data; caps papers for agent presentations	Safe presentations
agents-only	Skip literature; rescore existing DB	Team Supabase dashboard

scan	Incremental pull; skip known source_ids	Cheap updates
full	Live PubMed + trials + optional grants	Fresh research

End-to-End Example

Agent 3 finds: GBA-mutation PD -> lysosomal dysfunction -> GCase activation (8 papers, 1 trial). Agent 4: evidence_strength = 78. Agent 5: commercial_potential = 85. Agent 6: confidence = $(78 \times 0.55) + (85 \times 0.45) = 81.15$ -> Prioritize.

Trust & Safety

- No invented PMIDs/NCT ids - real source_id or DEMO-* placeholders only.
- Append-only agent_outputs trace per run_id.
- cli.py build blocked on live Supabase (truncates tables).
- Synthetic cohort in UI is generated, not stored PHI.

Part 2

Technical Summary

For developers familiar with Python, SQL, and REST APIs. Module paths and contracts.

Architecture

Blackboard pattern: agents in `neurodiscover/agents/` are decoupled modules. `orchestrator.py` routes pipeline modes, assigns `run_id`, persists agent return dicts to SQL, and logs `agent_outputs` after each step. Agents 2-6 do not import each other.

Layer map:

<code>ingestion/</code>	- <code>published_pull</code> , <code>biorexiv_pull</code> , <code>grants_pull</code> (no scoring)
<code>agents/</code>	- <code>literature_agent</code> , <code>patient_subgroup_agent</code> , ...
<code>orchestrator.py</code>	- <code>demo</code> <code>scan</code> <code>full</code> <code>agents-only</code>
<code>api_server.py</code>	- FastAPI; read-only queries for dashboard
<code>frontend/index.html</code>	- vanilla JS; polls <code>/api/run-status</code>
<code>db.py</code>	- SQLite or Postgres via <code>SUPABASE_DATABASE_URL</code>

Pipeline Sequence

A1 Literature -> A2 Subgroups -> A3 Connections -> A4 Evidence -> A5 Commercial -> A6 Conclusions

Agent 1 may run as subprocess (`demo/scan` via `cli.py`) or in-process (`full` mode). Agents 2-6 always in-process. Orchestrator commits after each agent for live stepper polling.

Agent I/O Contract

#	Module	Reads	Writes
1	<code>literature_agent.py</code>	BioMCP, RePORTER	<code>evidence</code> , <code>subgroup_evidence</code> , <code>scan_state</code>
2	<code>patient_subgroup_agent.py</code>	<code>evidence</code> (subgroup NOT NULL)	<code>subgroups</code>
3	<code>treatment_connection_agent.py</code>	<code>evidence</code> + <code>subgroups</code> (in-memory)	<code>treatment_connections</code> , <code>connection_evidence</code>
4	<code>evidence_scoring_agent.py</code>	<code>connections</code> + matching <code>evidence</code>	UPDATE <code>evidence_strength</code>
5	<code>commercial_discovery_agent.py</code>	<code>scored_connections</code> in-memory	UPDATE <code>commercial_potential</code>
6	<code>orchestrator.py</code> (ranking)	<code>scored treatment_connections</code>	<code>recommendations</code>

Agent 1 - Literature

CLI entrypoints:

<code>python3 cli.py pull --disease 'Parkinson disease' --max 150</code>
<code>python3 cli.py pull --include-preprints 20 --with-fulltext</code>
<code>python3 cli.py pull-grants --limit 30</code>
<code>python3 cli.py scan --max 5 # incremental; skips LLM for known source_ids</code>

`EXTRACT_BACKEND=nebius|ollama|none` for structured extraction. `UNIQUE(source_type, source_id)`

dedups.

Key evidence columns: source_type, source_id, subgroup, mechanism, treatment, key_result, access_type.

Agent 4 - Evidence Scoring Heuristics

Pre-scale 0-10 in Python; orchestrator _scale_score() x10 -> DB 0-100:

```
base = 4.0 + min(4.5, evidence_count * 0.22)
+0.6 if literature present; +0.8 if trial; +0.4 if grant; cap 10.0
```

Agent 5 - Commercial Heuristics

```
base = 5.5 + min(2.5, evidence_count * 0.12)
+0.9 GBA/LRRK2 subgroup; +0.6 alpha-synuclein; +0.7 lysosomal/kinase; +0.6 if treatment
set
```

Agent 6 - Ranking

```
confidence = evidence_strength * 0.55 + commercial_potential * 0.45
INSERT INTO recommendations (run_id, connection_id, subgroup, treatment, confidence,
tier, rationale)
```

CUA optional path: python -m cua.nih.run_db --db <url|path> -> cua/outputs/*.json|html (SELECT-only).

Database

Schema source of truth: neurodiscover/schema.sql (SQLite), schema.pg.sql (Postgres).

Local: neurodiscover/neurodiscover.db (gitignored). Team: SUPABASE_DATABASE_URL session pooler.

Never run cli.py build on Supabase - truncates all tables. Use scan/pull to add evidence.

Validate after schema changes: python3 cli.py validate

Orchestrator & API

Entry: POST /api/run-discovery (or make dashboard -> cli.py dashboard).

Run status (run_status.py): Complete = 6 agents in trace + >=1 recommendation.

Dashboard launcher: FastAPI :5000, static UI :8080. Supabase path uses agents-only on startup.

Key Files

```
neurodiscover/orchestrator.py
neurodiscover/pipeline_mode.py
neurodiscover/api_server.py
neurodiscover/cli.py
docs/developer-reference/agent-io.md
docs/developer-reference/database.md
docs/workflow/index.md
```

Quick Start (Developer)

```
cd neurodiscover && pip install -r requirements.txt
cp .env.example .env # SUPABASE_DATABASE_URL optional
```

```
make dashboard          # local demo or Supabase agents-only
python3 cli.py pull --max 10  # add evidence locally
python3 cli.py query "SELECT COUNT(*) FROM evidence"
```

Scoring Data Flow

Agent 3 returns connection dicts -> orchestrator upserts treatment_connections + connection_evidence. Agent 4 updates evidence_strength per connection_id. Agent 5 updates commercial_potential. Agent 6 reads both scores, computes confidence, writes recommendations with shared run_id. Dashboard reads recommendations + agent_outputs WHERE run_id = latest.